

Builder Journal Week 2

AI Engineering Internship

Why did I choose AI Engineering?

I chose AI Engineering because it sits at the intersection of two things I enjoy most: building real, usable software and working with systems that reason over language and data rather than just storing and displaying it. Machine learning and NLP courses in my BS Artificial Intelligence program gave me the theory, but I wanted to move beyond notebooks and model training scripts toward building complete, end-to-end products that someone could actually open in a browser and use. AI Engineering felt like the right path because it combines model integration, backend engineering, and product thinking into a single discipline, which matches both my coursework and the practical, project-based way I prefer to learn.

What type of AI products do I want to build?

I am most interested in building applied AI products that solve concrete, everyday problems rather than pure research systems tools like retrieval-augmented assistants, computer vision applications, and automation systems that plug directly into a workflow someone already has. My prior projects (an eye-controlled mouse system, a face mask detection system, and an automated question generation tool) all share this pattern: take an underlying AI capability and wrap it in a product people can actually interact with. Going forward, I want to keep building in this direction — practical AI systems with real interfaces, real data pipelines, and real user-facing value, rather than models that only exist as an offline evaluation script.

What was my biggest learning this week (Week 2)?

The biggest learning this week was understanding Retrieval-Augmented Generation (RAG) as a full system rather than a single concept. Going in, I understood embeddings and language models individually, but building the project end-to-end showed me how many moving parts have to work together correctly: consistent embedding models between ingestion and query time, chunking strategy directly affecting retrieval quality, and prompt construction determining whether the model stays grounded or starts guessing. I also learned that a vector database (ChromaDB) and a relational database (MySQL) serve completely different purposes in the same system — one finds semantically similar content, the other manages structured records and a real RAG application needs both working together, not one replacing the other.

What challenges did I face?

- Environment and configuration issues missing Python packages, and MySQL running on a non-default port (3307), which caused connection errors until traced through the full error traceback.
- LLM integration issues an outdated model name returned a 404 error, and a default max_tokens value exceeded available API credits, returning a 402 error.

- A retrieval scoping bug where a new chat session answered questions using a previously uploaded document instead of the one just uploaded, because all documents shared a single global vector collection.
- Frontend/backend integration issues, such as a confusing JavaScript parsing error that was actually caused by an unhandled backend exception returning an HTML error page instead of JSON.

How did I solve them?

For each issue, the approach was the same: read the full error traceback rather than only the surface-level browser error, since the real cause was almost always visible further down the stack trace. Errors were fixed one at a time and re-tested immediately, since later errors were sometimes hidden behind earlier ones. For the document-scoping bug specifically, I restructured the data model to link uploaded documents to a specific chat session, and updated the retrieval query to filter by the documents attached to that session instead of searching the entire knowledge base by default. I also improved frontend error handling to check the response status before parsing JSON, so future backend errors would surface as a readable message instead of a cryptic exception.

What are my goals for Week 3?

- Add authentication so documents and chat history are properly scoped per user.
- Write automated tests for the upload and chat routes to catch regressions instead of relying only on manual testing.
- Deploy the application to a live host so it can be accessed and demonstrated outside of localhost.
- Explore more advanced retrieval techniques (hybrid search, re-ranking) to improve answer quality on longer, multi-topic documents.
- Continue strengthening my understanding of prompt engineering and hallucination-reduction techniques, building on this week's experiments.